Mappingg Developer API — Complete Guide

Oct 14, 2026 · @Tushar Sontakke

Overview

The Mappingg Developer API lets an approved partner's system read Mappingg's public real-estate data, and optionally send buyer enquiries into the Mappingg CRM, using a private API key that you issue and can switch off at any time.

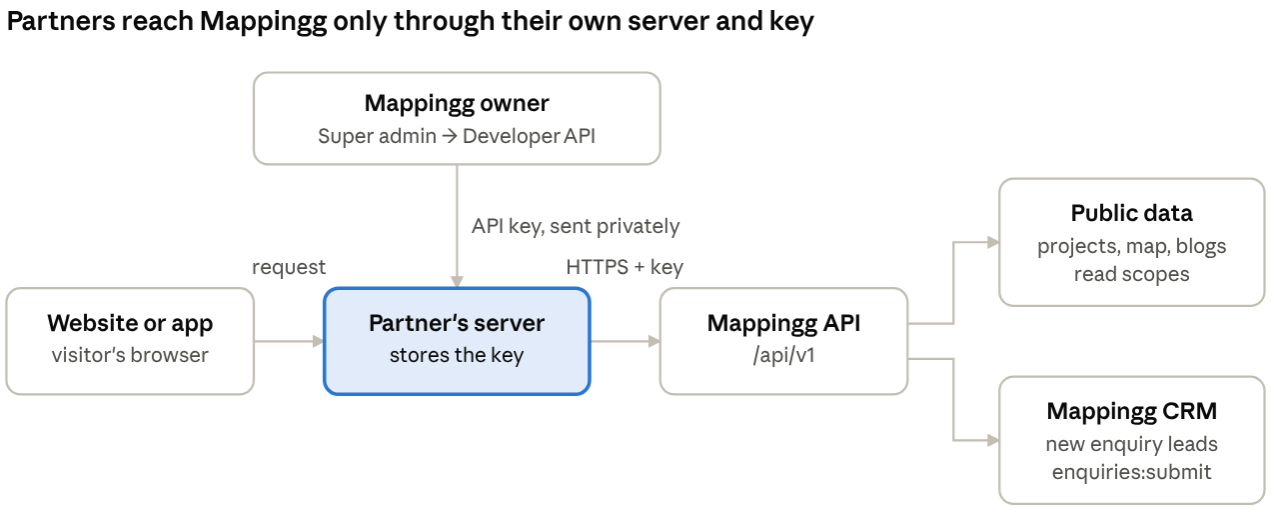
What a partner can get

- **Projects:** name, developer, locality, city, configuration, published price, possession, MahaRERA number where listed, coordinates, image, page link.
- **Developers and localities:** lists with project counts, and each one's projects.
- **Map data:** project pins inside a map area (clustered when zoomed out) and infrastructure markers such as metro stations and hospitals.
- **Blogs:** published articles only.
- **Enquiries (optional, write):** send a visitor's enquiry from their website into your CRM.

What a partner can never get: buyer or customer details, leads or enquiries already in your CRM, accounts, drafts, hidden or pending projects, uploaded files, the admin panel or the database. A field you hide on a project's public card is hidden in the API too.

How to use this guide

- **Part A** (next section) is for you, the Mappingg owner: how to create access and hand it over.
- **Part B onward** is for the partner's developer. You can share this whole document with them; Part A tells them nothing secret.
- The live versions of the partner docs are always at mappingg.com/developer-api, with the full technical reference at mappingg.com/developer-api/reference.



how a partner connects · owner, partner server, Mappingg API

The key only ever travels from you to the partner's server; their website or app talks to their server, never to Mappingg directly.

Part A — For the Mappingg owner: create and share access

You give a partner access by creating one **application** for them in Super admin; the application holds their permissions, limits and API key. Only the owner account can do this.

Step 1 — Collect the partner's details

Before creating anything, get these from the partner (by email is fine):

Detail	Why you need it
Organisation name	Shown in your admin and usage reports
Technical contact email	Where you send the docs; used if a key must be rotated
Website (domain)	Required for sending enquiries: they're only accepted from pages on this domain
What they want to build	Decides which scopes to grant
Expected traffic	Decides the rate limit and daily quota

Step 2 — Create the application

1. Sign in as the owner and open **Super admin → Developer API** (`/dashboard/s-admin` , the *Developer API* tab).
2. Click **New application** and fill in name, organisation, contact email, website and purpose.
3. Tick **only the scopes they need** (table below). Nothing is granted unless you tick it.
4. Set the limits. The defaults suit most partners: 60 requests a minute, 5,000 a day, 50 records a page.
5. Optional: set an **expiry date** for trials or fixed-term contracts.
6. Click **Create & generate key**.

Scope	Lets the partner	Grant when
`projects:read`	List and read public projects, plus categories	Almost always
`developers:read`	List developers and their projects	They show builder pages
`localities:read`	List localities and their projects	They show area pages
`map:read`	Get project pins inside a map area	They show a map
`infrastructure:read`	Get metro, hospital and other markers	They show amenities on a map
`blogs:read`	Read published articles	They republish your content (with attribution)
`enquiries:submit`	**Send enquiries into your CRM**	Only for trusted partners under a written agreement covering visitor consent

If you tick `enquiries:submit` , two more limits appear: enquiries per minute (default 10) and per day (default 200).

Step 3 — Copy the key (shown once)

A yellow box shows the key, which looks like `mgk_1a2b3c4d5e6f7a8b_…` (64 characters). **Copy it now.** Mappingg stores only a scrambled fingerprint of it, so nobody, including you, can see it again. If it's lost, rotate the key (Step 6).

Step 4 — Share it safely

- Send the **key** through a private channel: a password-manager share (1Password, Bitwarden), or a one-time secret link. Avoid plain email, WhatsApp or chat if you can.
- Send **this document** (or the link to mappingg.com/developer-api) separately.
- Tell them their **application ID** (`app_…`, shown on the application page). They quote it when they contact support. It isn't secret.
- Remind them: the key goes on their **server only**, never in a website or mobile app.

Step 5 — Watch usage

Open the application in the Developer API tab to see requests today and over 7 days, errors, average response time, enquiries accepted and rejected, the last time the key was used, recent requests and the audit trail. **Usage CSV** downloads a per-day report without any personal data. The list page also shows **security events**: failed keys, blocked clients and rate limits.

Step 6 — Change, pause or end access

You want to	Do this	Effect
Change scopes or limits	**Edit scopes & limits**	Next request
Replace a key (staff change, suspected leak, routine)	**Rotate key**, choose an overlap in hours	New key shown once; old key keeps working for the overlap, then stops. 0 = stops now
Stop one key	**Revoke** next to the key	Immediately
Pause the partner	**Disable**	Immediately, reversible with **Enable**
End the partnership	**Revoke** the application	Immediately and permanently; every key dies
Switch the whole API off	Set `DEVELOPER_API_DISABLED=1` on the server and restart	All endpoints except `/health` return 503

Every change is recorded in the application's audit trail with who did it and when.

Part B — Quick start for the partner developer

You can make your first successful call in about five minutes with the key and application ID that Mappingg sent you.

1. **Check the service is up (no key needed):**

   ```
   curl https://www.mappingg.com/api/v1/health
   ```

   You should get `{"success":true,"data":{"status":"ok",…}}`.
2. **See real sample data (no key needed):** open https://www.mappingg.com/api/v1/sample/projects in a browser. It returns three projects so you can see the response shape.
3. **Store your key on your server**, for example in an environment variable called `MAPPINGG_API_KEY`. Never put it in website or app code, or in a code repository.
4. **Check your key and permissions:**

   ```
   curl https://www.mappingg.com/api/v1/metadata \
     -H "Authorization: Bearer $MAPPINGG_API_KEY"
   ```

   The response lists your application, the scopes you were granted and your limits.
5. **Make a real call**, for example the first 10 under-construction projects in Kharadi:

   ```
   curl "https://www.mappingg.com/api/v1/projects?
   locality=kharadi&status=under_construction&limit=10" \
     -H "Authorization: Bearer $MAPPINGG_API_KEY"
   ```

If a step fails, the response's `error.code` tells you why. See **Errors and status codes** below.

Authentication and key security

Every request except `/health`, `/openapi.json` and `/sample/projects` must carry your key in the `Authorization` header:

```
Authorization: Bearer
mgk_xxxxxxxxxxxxxxx_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

- **Only this header works.** A key in the URL, a query string, a cookie or any other header is ignored and you get `401 missing_api_key`.
- **Server-side only.** Anything in a web page or mobile app can be copied. The keyed endpoints send no CORS headers, so browsers can't call them directly anyway. Your website should call *your* backend, and your backend calls Mappingg.

- **Keep it secret:** store it in an environment variable or a secrets manager; never commit it to Git, paste it in tickets or log it. Mappingg never logs keys either.
- **One key per integration.** If you run two separate systems, ask for two applications so one can be switched off without breaking the other.
- **Rotation:** when Mappingg rotates your key you receive a new one, and the old one keeps working for an agreed overlap (usually 24 hours). Deploy the new key within that window.
- **If a key leaks:** email Mappingg straight away with your application ID. The key is revoked at once and a new one issued.
- **Repeated wrong keys** from the same network are blocked for 10 minutes (`429 client_blocked`) after 30 failures in 10 minutes.

Base URL, versioning and response format

All requests go to `https://www.mappingg.com/api/v1` over HTTPS, and every response is JSON in one of two shapes.

Success

```
{
  "success": true,
  "data": [ … ],
  "pagination": { "page": 1, "limit": 20, "total": 57, "hasMore": true },
  "meta": { "requestId": "6f1c2a9e-…", "apiVersion": "v1" }
}
```

`pagination` appears only on list endpoints. `data` is an object for single items.

Error

```
{
  "success": false,
  "error": {
    "code": "insufficient_scope",
    "message": "This key is not allowed to use this endpoint (needs scope
\"blogs:read\").",
    "details": { … },
    "requestId": "6f1c2a9e-…"
  }
}
```

Build your error handling on `error.code` (stable), not on `message` (may be reworded). `details` appears only for validation errors.

Useful response headers

Header	Meaning
`X-Request-Id`	Unique ID of this request; quote it to support
`X-Api-Version`	Always `v1` here
`X-RateLimit-Limit` / `-Remaining` / `-Reset`	Per-minute limit, what's left, and when it resets (Unix seconds)
`X-Quota-Limit` / `-Remaining` / `-Reset`	Daily quota, what's left, and when it resets (00:00 UTC)
`Retry-After`	On a 429: seconds to wait before retrying

Versioning: the version is in the path. Within v1 Mappingg only adds things: new endpoints, new optional parameters and new response fields. Ignore fields you don't recognise. Breaking changes go into a new version, and v1 keeps working for at least six months after the new version is announced to your contact email.

Formats: dates are ISO 8601 (`2026-10-14T05:30:00.000Z`). Coordinates are decimal degrees. Prices, configurations and possession dates are the text as published on the project page, for example `"₹ 1.12 Cr Onwards"` ; they are not converted to numbers.

Endpoint reference

There are 16 endpoints; all are `GET` except `POST /enquiries` , and each needs the scope shown.

Endpoint	Scope	Returns
`GET /health`	none	Service status: `ok` or `degraded`
`GET /openapi.json`	none	This API as an OpenAPI 3.1 file (import into Postman or Insomnia)
`GET /sample/projects`	none	Three sample projects
`GET /metadata`	any valid key	Your application, scopes and limits
`GET /projects`	`projects:read`	Public projects, filtered and paged
`GET /projects/{id}`	`projects:read`	One project by number (`135`) or slug (`new-beginnings-kharadi-135`)

Endpoint	Scope	Returns
`GET /categories`	`projects:read`	Statuses, property types and cities, with project counts
`GET /developers`	`developers:read`	Developers with public projects
`GET /developers/{id}`	`developers:read`	One developer and up to 100 of their projects
`GET /localities`	`localities:read`	Localities with public projects
`GET /localities/{id}`	`localities:read`	One locality and up to 100 of its projects
`GET /map/pins`	`map:read`	Project pins inside a map area
`GET /map/infrastructure`	`infrastructure:read`	Metro, hospital, school and other markers
`GET /blogs`	`blogs:read`	Published articles, newest first
`GET /blogs/{slug}`	`blogs:read`	One published article with its HTML
`POST /enquiries`	`enquiries:submit`	Sends an enquiry into the Mappingg CRM (see its own section)

Query parameters

Endpoint	Parameters
`/projects`	`page`, `limit`, `q` (name search, 2–80 chars), `locality`, `developer`, `type` (IDs from the other endpoints), `city` (`pune`, `mumbai`, `nashik`, `hyderabad`, `bengaluru`, `delhi`, `dubai`), `status` (`upcoming`, `under_construction`, `ready_to_move`, `sold`), `updated_since` (ISO date-time with time zone), `sort` (`id`, `name`, `-updated_at`)
`/developers`, `/localities`	`page`, `limit`, `q`, `city`
`/map/pins`	`north`, `south`, `east`, `west` (required), `zoom` (0–22; below 13 nearby pins are grouped into clusters), `limit` (up to 500, default 200), `status`, `type`, `developer`, `locality`

Endpoint	Parameters
`/map/infrastructure`	`page`, `limit`, `type` (e.g. `metro`, `hospital`), optional `north`, `south`, `east`, `west` (all four or none)
`/blogs`	`page`, `limit` (default 10), `tag`

Unknown or repeated parameters are rejected with `400 invalid_parameter`, so a typo never silently returns everything.

Project fields

Field	Example	Notes
`id`	`135`	Stable project number; use it as your key
`slug`, `url`	`new-beginnings-kharadi-135`, full mappingg.com link	Link back to the project page
`name`	`New Beginnings`	
`developer`	`{ "id": "venkatesh-buildcon", "name": "Venkatesh Buildcon" }`	
`property_type`	`Residential`	
`status`, `status_label`	`under_construction`, `Under construction`	
`configuration`	`2 / 3 / 4 BHK`	As published
`price`	`₹ 1.12 Cr Onwards`	As published, not a number
`possession`	`Dec 2030`	As published
`locality`, `city`	`{ "id": "kharadi", "name": "Kharadi" }`, `{ "id": "pune", "name": "Pune" }`	
`coordinates`	`{ "lat": 18.55, "lng": 73.94 }`	Can be null
`image_url`	full URL	Can be null

Field	Example	Notes
`updated_at`	ISO date-time	When the project last changed
Single project only: `description`, `highlights`, `address`, `area_sqft`, `launch_date`, `rera_number`, `video_url`		`rera_number` is as listed by the developer; Mappingg doesn't certify it, so verify it on MahaRERA

Any field can be `null`. That happens when it's missing or when Mappingg hides it on the public project card.

Other response fields

- **Developer / locality:** `id`, `name`, `project_count`, `url`, `cities`; developers also have `localities`; localities also have `center` (the average of their projects' coordinates, a map centre, not a boundary). Single items add `projects` and `projects_truncated`.
- **Map pins:** `bounds`, `zoom`, `total`, `truncated`, `clusters` (`lat`, `lng`, `count`) and `pins` (`id`, `slug`, `url`, `name`, `developer`, `status`, `property_type`, `locality`, `coordinates`).
- **Infrastructure:** `id`, `name`, `type`, `type_label`, `coordinates`, `updated_at`.
- **Blogs:** `id`, `slug`, `url`, `title`, `excerpt`, `description`, `tags`, `author`, `cover_image_url`, `reading_minutes`, `published_at`, `updated_at`; a single article adds `content_html`.

The exact schema for every field is in the API reference.

Pagination, caching and keeping data in sync

List endpoints return one page at a time: ask for the next page while `pagination.hasMore` is `true`.

- `page` starts at 1. `limit` defaults to 20 (10 for blogs, 50 for infrastructure) and can't exceed your application's page size (usually 50, at most 100).
- Paging stops at record 5,000. Beyond that, narrow the query with filters (`city`, `locality`, `status` …) instead of paging further.
- `pagination.total` is the number of records matching your filters.

Freshness: data comes from the live site with a short server cache, so changes show up within about a minute.

Caching on your side: responses carry `Cache-Control: private, max-age=30` and `Vary: Authorization`. You may cache them in your own backend, but never in a shared or public cache (a CDN, a proxy shared with other customers).

Recommended sync pattern

1. **First load:** page through `/projects` once and store each project by its `id`.
2. **Every 15–60 minutes:** call `/projects?updated_since=<time of your last sync>&sort=-updated_at` and update what changed.
3. **Once a day:** check the projects you show still exist. A project that has been unpublished disappears from lists, and `/projects/{id}` returns `404`. Remove those from your site.
4. Store the `url` with each project, and link back to it (see **Permitted use**).

Sending enquiries into the Mappingg CRM

With the `enquiries:submit` scope, your **server** can send a visitor's property enquiry to `POST /enquiries`; it becomes a lead in the Mappingg CRM, credited to your website, and the Mappingg team follows it up.

Before you start: Mappingg must have your website's domain on your application. Enquiries are accepted only when `sourcePage` is an https page on that domain or one of its subdomains.

Request

```
POST https://www.mappingg.com/api/v1/enquiries
Authorization: Bearer <your key>
Content-Type: application/json
```

```
{
  "submissionId": "9f0c2b6e-1d7a-4b55-9a3e-2f6a1c8d4e01",
  "name": "Asha Patil",
  "phone": "+91 9876543210",
  "email": "asha@example.com",
  "projectId": 135,
  "bhk": "2 BHK",
  "budget": "₹1–1.5 Cr",
  "message": "Please share the price sheet.",
  "consent": {
    "enquiry": true,
    "whatsapp": true,
    "text": "I agree to be contacted by Example Realty and Mappingg about
this enquiry. I'd also like WhatsApp updates on matching projects.",
    "textVersion": "2026-10",
    "capturedAt": "2026-10-14T10:15:00+05:30"
  },
  "sourcePage": "https://www.example-realty.com/kharadi/new-projects",
  "campaign": { "source": "google", "medium": "cpc", "name": "kharadi-oct" }
}
```

Field	Required	Rules
`submissionId`	yes	Your unique ID for this enquiry, 8–64 letters, digits, `-` or `_`. A UUID is ideal. Reuse it when retrying
`name`	yes	2–120 characters, must contain letters
`phone`	yes	Country code + 10 digits, e.g. `+91 9876543210`; a plain 10-digit number is read as +91
`email`	no	Valid email
`projectId`	no	A published project's `id` from `/projects`
`projectName`	no	If sent with `projectId`, it must match that project
`locality`, `budget`	no	Up to 120 and 40 characters
`bhk`	no	One of: 1 RK, 1 BHK, 1.5 BHK, 2 BHK, 2.5 BHK, 3 BHK, 3.5 BHK, 4 BHK, 4.5 BHK, 5+ BHK, Villa, Plot, Shop, Office
`message`	no	Up to 1,000 characters
`consent`	yes	See below

Field	Required	Rules
`sourcePage`	yes	The https page on your registered website where the form was. The query string isn't stored
`campaign`	no	`source`, `medium`, `name`, `term`, `content`

Any other field, such as `status`, `assigned_to`, `id` or `notes`, is rejected with `400`. You can't decide how Mappingg handles the lead. The whole body may be at most 16 KB.

Consent: your responsibility

- Show the visitor a clear statement next to an **unticked** checkbox, and send the enquiry only if they ticked it.
- `consent.enquiry` must be `true`: the visitor asked to be contacted about this enquiry.
- `call` and `whatsapp` are separate, optional opt-ins for other matching offers. Send `true` only if the visitor ticked those boxes too.
- Send the **exact wording** shown in `consent.text` (at least 20 characters), your label for that wording in `textVersion` (change it whenever the wording changes), and the submit time in `capturedAt`. It must be within the last 24 hours.
- Mappingg can't see your form, so it records this as consent **stated by you**, with your wording, version and time. Receiving `"enquiry": true` doesn't prove the visitor agreed. Sending enquiries without real consent breaks the terms and gets your access revoked.
- Your privacy notice must say that enquiries are shared with Mappingg (Associatte PropTech Pvt Ltd) for follow-up, and link to mappingg.com/privacy.

Responses, retries and duplicates

Response	Meaning	What to do
`201` `{"submissionId":"…","reference":"enq_…","status":"received"}`	Received	Store the reference for support
`200` + header `Idempotent-Replayed: true`	Same `submissionId` and same body as before	Nothing; it's the original receipt, no second lead

Response	Meaning	What to do
`409 submission_id_reused`	Same `submissionId`, different body	Use a new `submissionId` for each new enquiry
`409 submission_in_progress`	The first attempt is still being saved	Retry after 2 seconds
`400`, `413`, `415`, `422`	The input is wrong	Fix it; don't retry unchanged
`429`	A limit was hit	Wait `Retry-After` seconds
`5xx` or no response	Our side or the network	Retry with the **same** `submissionId` and body

- If the same phone number asks about the same project again within 24 hours, it is added to the existing lead rather than creating a new one. The response looks the same either way: the API never reveals whether a person is already known to Mappingg.
- The API can't read, list, change or delete enquiries.

Limits

Enquiries have their own per-application limits (default 10 a minute and 200 a day, UTC) on top of the general request limits. Rejected submissions count too. After 50 rejected submissions in an hour, enquiries pause for the rest of that hour (`429 enquiries_paused`). Rate-limit your own form per visitor as well, so bots can't use up your quota.

What happens after

The enquiry appears in the Mappingg CRM as a new lead marked "via partner website <your domain>", is sent to Mappingg's sales system and is followed up by the Mappingg team. To withdraw an enquiry or a visitor's consent, email Mappingg with the `reference`.

Errors and status codes

Every error has a stable `error.code`; handle errors by that code, and quote the `requestId` if you contact support.

Status	`error.code`	Meaning	What to do
400	`invalid_parameter`	Unknown, repeated or out-of-range query parameter, or a bad ID in the URL	Fix the request; see `details`
400	`invalid_request`	Enquiry body has a missing, invalid or unknown field	Fix the field named in `details`
400	`invalid_json`	Body isn't valid JSON	Fix the JSON
401	`missing_api_key`	No `Authorization: Bearer` header	Add the header
401	`invalid_api_key`	Key is wrong or mistyped	Check the stored key
401	`revoked_api_key`	Key was revoked	Ask Mappingg for a new key
401	`expired_api_key`	Key expired, e.g. after a rotation	Switch to your new key
403	`insufficient_scope`	Key lacks the scope for this endpoint	Ask Mappingg to add the scope
403	`application_disabled`	Access paused by Mappingg	Contact Mappingg
403	`application_revoked`	Access ended permanently	Contact Mappingg
403	`application_expired`	Access end date has passed	Ask Mappingg to extend it
404	`not_found`	No such project, developer, locality, article or endpoint	Remove it from your data
405	(no body)	Wrong HTTP method	Use the method in the reference

Status	`error.code`	Meaning	What to do
409	`submission_id_reused`	Enquiry `submissionId` already used for a different enquiry	Use a new `submissionId`
409	`submission_in_progress`	Same enquiry still being saved	Retry after 2 seconds
413	`payload_too_large`	Enquiry body over 16 KB	Shorten it
415	`unsupported_media_type`	Enquiry not sent as JSON	Send `Content-Type: application/json`
422	`invalid_phone`	Phone isn't country code + 10 digits	Fix the number
422	`invalid_consent`	`capturedAt` is in the future or over 24 hours old	Send enquiries when they happen
422	`website_not_registered`	Your application has no website set	Ask Mappingg to add your domain
422	`source_not_allowed`	`sourcePage` isn't on your registered domain	Use the real page URL on your domain
422	`project_not_found`	`projectId` isn't a published project	Use an ID from `/projects`, or leave it out
422	`project_mismatch`	`projectName` doesn't match `projectId`	Send one, or the right pair
422	`content_rejected`	Looks like spam (links in the name, many links in the message)	Don't forward spam
429	`rate_limited`	Per-minute or concurrency limit hit	Wait `Retry-After` seconds
429	`quota_exceeded`	Daily quota used up	Wait until 00:00 UTC, or ask for more
429	`client_blocked`	Too many wrong keys from your network	Fix the key; wait 10 minutes
429	`enquiries_paused`	50 rejected enquiries in an hour	Fix the integration; wait for the next hour

Status	`error.code`	Meaning	What to do
500	`internal_error`	Problem on Mappingg's side	Retry later; report the `requestId` if it persists
500	`response_too_large`	Response would be too big	Use a smaller `limit`
503	`api_disabled`	API temporarily switched off	Retry later
504	`timeout`	Request took over 10 seconds	Retry, with narrower filters

Retry rule: retry only `429` (after `Retry-After`), `500`, `503`, `504` and network errors, waiting longer each time (for example 1, 2, then 4 seconds). Never retry a `4xx` other than 429 without changing the request.

Rate limits and quotas

Each application has its own limits, set by Mappingg; your current values are in `GET /metadata` and in the `X-RateLimit-*` and `X-Quota-*` headers of every response.

Limit	Default	Maximum Mappingg can set
Requests per minute	60	600
Requests per day (resets 00:00 UTC)	5,000	200,000
Records per page	50	100
Records reachable by paging	5,000	5,000
Requests in flight at once	5	5
Map pins per request	200 (default), 500 (max)	500
Enquiries per minute	10	120
Enquiries per day	200	5,000
Request timeout	10 seconds	—
Calls without a key (`/health`, `/openapi.json`, `/sample/projects`)	30 per minute per network	—

Every request counts, including ones that fail. Over a limit you get `429` with `Retry-After`. To stay well inside the limits, cache responses on your server and sync changes with `updated_since` instead of downloading everything again. Need higher limits? Ask Mappingg, saying what you're building and the traffic you expect.

Code examples

Each example reads the key from the `MAPPINGG_API_KEY` environment variable and runs on a server.

cURL

```
# Projects in Pune, newest changes first
curl "https://www.mappingg.com/api/v1/projects?city=pune&sort=-
updated_at&limit=20" \
  -H "Authorization: Bearer $MAPPINGG_API_KEY"

# One project
curl https://www.mappingg.com/api/v1/projects/135 -H "Authorization: Bearer
$MAPPINGG_API_KEY"

# Pins in a map area (zoomed in, so no clusters)
curl "https://www.mappingg.com/api/v1/map/pins?
north=18.60&south=18.50&east=73.98&west=73.88&zoom=14" \
  -H "Authorization: Bearer $MAPPINGG_API_KEY"
```

Node.js (18 or later): fetch every project, with retries

```
const BASE = 'https://www.mappingg.com/api/v1';
const headers = { Authorization: `Bearer ${process.env.MAPPINGG_API_KEY}` };
const sleep = (s) => new Promise((r) => setTimeout(r, s * 1000));

async function get(path, attempt = 1) {
  const res = await fetch(BASE + path, { headers });
  if ((res.status === 429 || res.status >= 500) && attempt < 4) {
    await sleep(Number(res.headers.get('retry-after')) || 2 ** attempt);
    return get(path, attempt + 1);
  }
  const body = await res.json();
  if (!body.success) throw new Error(`${body.error.code}:
${body.error.message} (${body.error.requestId})`);
  return body;
}
```

```
async function allProjects(filters = 'city=pune') {
  const out = [];
  for (let page = 1; ; page++) {
    const body = await get(`/projects?${filters}&limit=50&page=${page}`);
    out.push(...body.data);
    if (!body.pagination.hasMore) return out;
  }
}

allProjects().then((ps) => console.log(ps.length, 'projects'));
```

Python (requests)

```
import os, time, requests

BASE = "https://www.mappingg.com/api/v1"
HEADERS = {"Authorization": f"Bearer {os.environ['MAPPINGG_API_KEY']}"}

def get(path, params=None):
    for attempt in range(1, 5):
        r = requests.get(BASE + path, headers=HEADERS, params=params,
timeout=15)
        if r.status_code == 429 or r.status_code >= 500:
            time.sleep(int(r.headers.get("Retry-After", 2 ** attempt)))
            continue
        body = r.json()
        if not body["success"]:
            raise RuntimeError(f"{body['error']['code']}: {body['error']
['message']}")
        return body
    raise RuntimeError("Mappingg API unavailable, try later")

for p in get("/projects", {"locality": "kharadi", "limit": 20})["data"]:
    print(p["id"], p["name"], p["price"], p["url"])
```

PHP (cURL)

```
<?php
function mappingg_get(string $path): array {
    $ch = curl_init('https://www.mappingg.com/api/v1' . $path);
    curl_setopt_array($ch, [
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_TIMEOUT => 15,
        CURLOPT_HTTPHEADER => ['Authorization: Bearer ' .
getenv('MAPPINGG_API_KEY')],
    ]);
    $body = json_decode(curl_exec($ch), true);
    curl_close($ch);
    if (empty($body['success'])) {
        throw new Exception(($body['error']['code'] ?? 'network_error') . ':
' . ($body['error']['message'] ?? ''));
    }
    return $body;
}

foreach (mappingg_get('/blogs?limit=5')['data'] as $post) {
    echo $post['title'], ' - ', $post['url'], PHP_EOL;
}
```

Enquiry form: website → your server → Mappingg

The browser posts the form to **your own** backend; only your backend knows the key.

```
<form id="enquire">
  <input name="name" required> <input name="phone" required> <input
name="email" type="email">
  <label><input type="checkbox" name="agree"> I agree to be contacted by
Example Realty and Mappingg about this enquiry.</label>
  <label><input type="checkbox" name="whatsapp"> I'd also like WhatsApp
updates on matching projects.</label>
  <button>Send</button>
</form>
<script>
  document.getElementById('enquire').addEventListener('submit', async (e) =>
{
    e.preventDefault();
    const f = new FormData(e.target);
    await fetch('/enquire', {                // YOUR server, same website —
not Mappingg
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name: f.get('name'), phone: f.get('phone'),
email: f.get('email') || undefined,
        agree: f.get('agree') === 'on', whatsapp: f.get('whatsapp') === 'on'
}),
    });
  });
</script>
```

```
// Your server (Node.js + Express)
import express from 'express';
import { randomUUID } from 'node:crypto';

const app = express();
app.use(express.json({ limit: '16kb' }));
const CONSENT = 'I agree to be contacted by Example Realty and Mappingg about
this enquiry.';
const WHATSAPP = " I'd also like WhatsApp updates on matching projects.";

app.post('/enquire', async (req, res) => {
  const { name, phone, email, agree, whatsapp } = req.body;
  if (agree !== true) return res.status(400).json({ error: 'Please tick the
consent box.' });
  const body = {
    submissionId: randomUUID(),              // the same ID on every retry
of this enquiry
    name, phone, email,
    consent: {
```

```
      enquiry: true,
      whatsapp: whatsapp === true,
      text: CONSENT + (whatsapp === true ? WHATSAPP : ''),
      textVersion: '2026-10',
      capturedAt: new Date().toISOString(),
    },
    sourcePage: 'https://www.example-realty.com/contact',
  };
  for (let attempt = 1; attempt <= 3; attempt++) {
    const r = await fetch('https://www.mappingg.com/api/v1/enquiries', {
      method: 'POST',
      headers: { Authorization: `Bearer ${process.env.MAPPINGG_API_KEY}`,
'Content-Type': 'application/json' },
      body: JSON.stringify(body),
    }).catch(() => null);
    if (r && (r.status === 201 || r.status === 200)) return res.json({ ok:
true });
    if (r && r.status < 500 && r.status !== 429) return
res.status(400).json({ error: (await r.json()).error.message });
    await new Promise((s) => setTimeout(s, (Number(r?.headers.get('retry-
after')) || attempt) * 1000));
  }
  res.status(503).json({ error: 'Please try again in a minute.' });
});

app.listen(3000);
```

Postman or Insomnia: import `https://www.mappingg.com/api/v1/openapi.json` to get every endpoint ready to try, then set an `Authorization: Bearer` header.

Permitted use, data processing and support

Access is a permission Mappingg gives your application, and it can be paused or ended if these terms are broken.

You may: show Mappingg data to your users on your website, app, CRM or dashboards; combine it with your own data; cache it on your servers as described above.

You must:

- Show "Source: Mappingg" with a link to the project or page `url` wherever you display the data.
- Keep your copy reasonably fresh, and remove projects that Mappingg has unpublished.

- Tell users that project details come from developers and are for reference only, and that MahaRERA numbers should be verified on the MahaRERA website.
- Keep the key secret and on your server.

You must not: resell or redistribute the data as a dataset; try to copy the whole database, or get round the limits with several keys; present the data as your own verified data; send enquiries without the visitor's real consent; send data about anyone other than the enquirer, or sensitive personal data.

Enquiry data: for enquiries you send, you collect the data from the visitor and are responsible for having a lawful basis and a clear notice. Mappingg receives it to follow up the enquiry under its privacy policy. Pass on any withdrawal request to Mappingg with the enquiry `reference`.

The terms of use, privacy policy and disclaimer also apply.

Support: email mappingg.associatte@gmail.com with your application ID (`app_…`), the `requestId` of the failing call, the time and what you expected. Service status is at /api/v1/health. Report a leaked key or a security issue the same way, straight away.

Checklists

Tick these off for each partner so nothing is missed.

Owner: before and after handing over a key

- [ ] Partner's organisation, technical contact email, website domain and use case are written down
- [ ] Written agreement (or at least an email) accepting the permitted-use terms; for `enquiries:submit`, it also covers visitor consent
- [ ] Application created with **only** the scopes they need
- [ ] Website field set to their real domain (required for enquiries)
- [ ] Limits match their expected traffic; expiry date set for trials
- [ ] Key copied from the yellow box before closing it
- [ ] Key sent through a private channel; this guide and the application ID sent separately
- [ ] Partner confirmed a successful `/metadata` call
- [ ] First week: usage, errors and security events checked in the application page
- [ ] Calendar reminder to review access (and rotate the key) every 6–12 months, and when their staff change

Partner: before going live

- [ ] Key stored in a server environment variable or secrets manager, not in code, Git, a web page or a mobile app
- [ ] `GET /metadata` works and shows the scopes and limits you expect
- [ ] Errors handled by `error.code`; `requestId` is logged on failures
- [ ] Retries only on 429, 5xx and network errors, honouring `Retry-After`
- [ ] Responses cached on your server; syncing uses `updated_since`; unpublished projects (404) are removed
- [ ] "Source: Mappingg" with a link back shown next to the data
- [ ] Enquiries only: the form has an unticked consent checkbox, and the exact wording is sent in `consent.text` with its `textVersion`
- [ ] Enquiries only: a new `submissionId` per enquiry, reused on retries; `sourcePage` is on your registered domain
- [ ] Enquiries only: your privacy notice mentions sharing enquiries with Mappingg
- [ ] Enquiries only: your own form is rate-limited per visitor
- [ ] A plan for key rotation: you can deploy a new key within 24 hours

Troubleshooting and FAQ

Most problems are the key, the scope or the website domain; check the `error.code` first.

Problem	Likely cause	Fix
`401 missing_api_key` with the key set	Key sent in the URL or another header, or the word `Bearer` missing	Send exactly `Authorization: Bearer <key>`
`401 invalid_api_key`	Key copied with a space or line break, or cut short	Copy the full 64-character key again
Worked yesterday, `401 expired_api_key` today	Key was rotated and the overlap ended	Deploy the new key Mappingg sent
`403 insufficient_scope`	Scope not granted	Ask Mappingg to add it
Browser shows a CORS error	Called from website JavaScript	Call from your server instead
`422 source_not_allowed`	`sourcePage` is on another domain, or `http://`	Use the https page on your registered domain, or ask Mappingg to update it

Problem	Likely cause	Fix
`422 invalid_phone`	Number isn't country code + 10 digits	Send `+91 9876543210` format
A field is `null`	Not filled in, or hidden on the public card	Show it as "not available"
A project disappeared	Unpublished on Mappingg	Remove it from your site
Frequent `429`	Too many calls	Cache, sync with `updated_since`, or ask for higher limits

FAQ

- **Is there a sandbox or test key?** No separate sandbox. Use `/sample/projects` (no key) to see the shape, then a key with read scopes. For enquiries, tell Mappingg before you send test enquiries so they aren't followed up.
- **Can I call the API from a mobile app?** Not directly. Put a small backend between the app and Mappingg, and keep the key there.
- **Do prices come as numbers?** No, they're the published text ("₹ 1.12 Cr Onwards"). Show them as text.
- **Are RERA numbers verified?** They're as listed by the developer. Verify on MahaRERA before relying on one.
- **Can I see the enquiries I sent?** Not through the API. Keep your own copy and the `reference`; Mappingg support can look one up by reference.
- **How do I get a new key if mine is lost?** Ask Mappingg to rotate it. The old key can be stopped at once or after an overlap.
- **Will the API change?** Only by adding things within v1. Breaking changes come as a new version, announced to your contact email at least six months ahead.